iptables: The Complete Breakdown

Architecture · Kernel Internals · Real Lab Implementations · Bug Hunting

From typing 'iptables -A INPUT' to watching the exact kernel function evaluate every packet

VirtualBox VM · Linux 5.11.1-tunable kernel · ~/linux source tree · Real dmesg & screenshots

This document is the complete story of iptables — every concept anchored in real commands, real kernel source modifications, and real outputs from my lab environment. Network namespace topology, live packet counters, iptables-from-source debug traces, kernel printk instrumentation, and the kernel panic from Bug #211911.

Dashed boxes mark spaces for your own additional output screenshots. All screenshots from my lab are embedded where they happened.

Part 1: What is iptables and Where Does It Live?

Most tutorials treat iptables as a magic firewall tool. It is not magic — it is a precise userspace interface to the Linux kernel's netfilter framework, a system of hooks wired directly into the packet processing code path. Understanding this distinction is the foundation of everything else in this document.

1.1 The Two-Layer Architecture

iptables is split across two layers. When you type a command, it is processed entirely in userspace first. The kernel only gets involved when your rule set is committed via setsockopt().

Layer	Component	Location	What It Does
Userspace	iptables binary	/usr/local/sbin/iptables (custom build)	Parses your command, calls libiptc
Userspace	libiptc	libip4tc.so	Builds binary rule blob, calls setsockopt()
Kernel	netfilter hooks	net/netfilter/core.c	Fires on every packet at 5 hook points
Kernel	ip_tables module	net/ipv4/netfilter/ip_tables.c	Walks your rules, evaluates matches per packet
Kernel	Match modules	net/netfilter/xt_string.c etc.	The actual matching logic: string, tcp, state…

Key Insight: Nothing Fires When You Add a Rule

When you run iptables -A INPUT -j DROP, absolutely nothing is evaluated. You are writing data into a kernel structure. The evaluation runs later, in ipt_do_table(), for every single packet that arrives. The rule is not a live filter — it is a specification that the kernel consults on every packet.

1.2 The Five Netfilter Hook Points

The kernel calls registered hook functions at exactly five points in the packet processing path. These map directly to iptables chain names. Every iptables rule you ever write fires at one of these five points.

```
  net/netfilter/core.c

# The 5 kernel hook points — where iptables rules are evaluated

NF_INET_PRE_ROUTING   hook #0  ===>  PREROUTING chain
NF_INET_LOCAL_IN      hook #1  ===>  INPUT chain       ← firewall: incoming
packets for this machine
NF_INET_FORWARD       hook #2  ===>  FORWARD chain     ← firewall: routed
packets
NF_INET_LOCAL_OUT     hook #3  ===>  OUTPUT chain      ← packets originating
from this machine
NF_INET_POST_ROUTING  hook #4  ===>  POSTROUTING chain ← SNAT / MASQUERADE

# Kernel function called at every hook point:
nf_hook_slow(skb, state, hook_entries, s)
  └─► iterates all registered hooks for this hooknum
        └─► calls ipt_do_table() for iptables rule evaluation
```

1.3 Tables, Chains, Rules — The Three Layers

Table	Available Chains	Primary Purpose
filter	INPUT FORWARD OUTPUT	Default table — ACCEPT or DROP. This is what people mean by 'firewall'
nat	PREROUTING OUTPUT POSTROUTING	Rewrite source/destination IP. Docker uses this for container NAT
mangle	All 5 chains	Modify packet headers: TTL, TOS, MARK fields
raw	PREROUTING OUTPUT	Bypass conntrack — for raw packet control or performance

Part 2: The Complete Journey of a Packet

Before writing a single rule, internalize this flow. Every iptables rule fires at exactly one node in this diagram. When something doesn't work, this map tells you where to look.

Full packet path through netfilter

```
# ──────────────────────────────────────────────────────────────────
# FULL PACKET PATH: NIC → netfilter hooks → application (or forwarded)
# ──────────────────────────────────────────────────────────────────

  [Network Interface — eth0 / lo / br0 / veth0]
          |
          ▼
  raw:PREROUTING        ← hook 0  (skip conntrack here if needed)
  mangle:PREROUTING     ← hook 0  (mark/modify packets)
  nat:PREROUTING        ← hook 0  (DNAT — change destination IP)
          |
          ▼
   [ ROUTING DECISION: is destination IP local to this machine? ]
        /                    \
   YES (local)                NO (forward to another host)
      |                          |
      ▼                          ▼
mangle:INPUT               mangle:FORWARD
filter:INPUT     ←         filter:FORWARD    ← YOUR FIREWALL RULES LIVE HERE
      |                          |
      ▼                          |
[Local Process]                  |
      |                          ▼
filter:OUTPUT              mangle:POSTROUTING
nat:OUTPUT                 nat:POSTROUTING   ← SNAT / MASQUERADE here
      |                          |
      └──────────────────────────▼
                  [Network Interface — outbound]
```

2.1 The Bridge Interface — How Our Lab Works

In our lab setup, a Linux bridge (br0) acts as the server interface. Client namespaces connect to it via veth pairs. When client1 sends a packet to the server at 192.168.100.1, it arrives on the bridge, which forwards it — hitting the INPUT chain rules on the server namespace exactly as a real server would.

Lab topology

```
# Our lab topology:

  [client1 ns]  10.0.0.11     ──────▶
  [client2 ns]  10.0.0.12     ──────▶  [br0 bridge]  192.168.100.1
  [client3 ns]  10.0.0.13     ──────▶            |
                                                 |
                                          filter:INPUT
                                     iptables rules evaluated here
                                      for EVERY incoming packet
```

Part 3: Lab Setup — The Network Namespace Topology

Before writing a single iptables rule, I built a controlled network environment using Linux network namespaces. This gives three isolated 'client machines' and a 'server' — all running inside one VirtualBox VM, all traffic going through real kernel networking stack and iptables hooks.

3.1 The Setup Script

setup-lab.sh creates the entire topology: a bridge interface acting as the server, and three client namespaces connected to it via veth pairs.

setup-lab.sh

```
#!/bin/bash
# setup-lab.sh — creates bridge server + 3 client namespaces

# Create bridge interface (this is the 'server')
sudo ip link add br0 type bridge
sudo ip addr add 192.168.100.1/24 dev br0
sudo ip link set br0 up

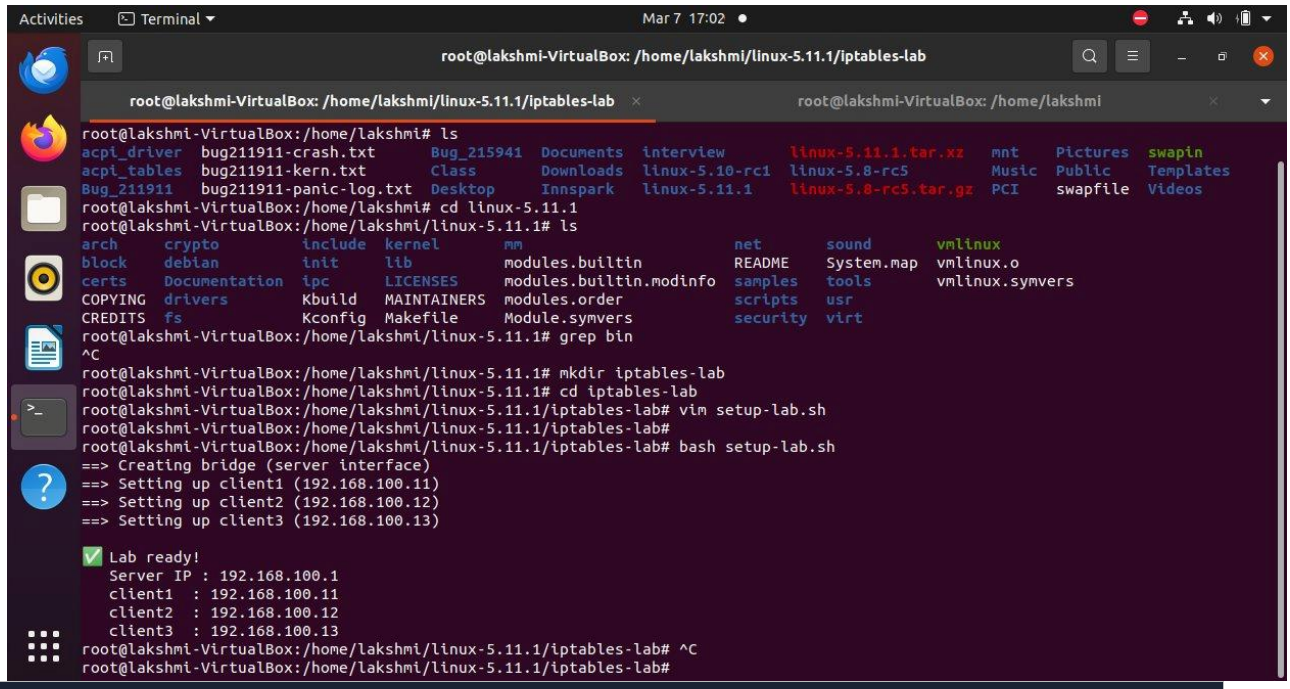
# Create each client namespace + veth pair connected to bridge
for i in 1 2 3; do
  sudo ip netns add client${i}
  sudo ip link add veth-c${i} type veth peer name veth-br${i}
  sudo ip link set veth-br${i} master br0
  sudo ip link set veth-br${i} up
  sudo ip link set veth-c${i} netns client${i}
  sudo ip netns exec client{i} ip addr add 192.168.100.1{i}/24 dev veth-c{i}
  sudo ip netns exec client${i} ip link set veth-c${i} up
done

# Set default route in each client
for i in 1 2 3; do
  sudo ip netns exec client${i} ip route add default via 192.168.100.1
done
```

STEP 1 **Run the setup script**

setup-lab.sh output

```
root@lakshmi-VirtualBox:/home/lakshmi/linux-5.11.1/iptables-lab# bash setup-
lab.sh
==> Creating bridge (server interface)
==> Setting up client1 (192.168.100.11)
==> Setting up client2 (192.168.100.12)
==> Setting up client3 (192.168.100.13)
☑ Lab ready!
   Server IP : 192.168.100.1
   client1   : 192.168.100.11
   client2   : 192.168.100.12
   client3   : 192.168.100.13
root@lakshmi-VirtualBox:/home/lakshmi/linux-5.11.1/iptables-lab#
```



$ bash setup-lab.sh

Lab ready: bridge server at 192.168.100.1, client1/2/3 at 192.168.100.11/12/13

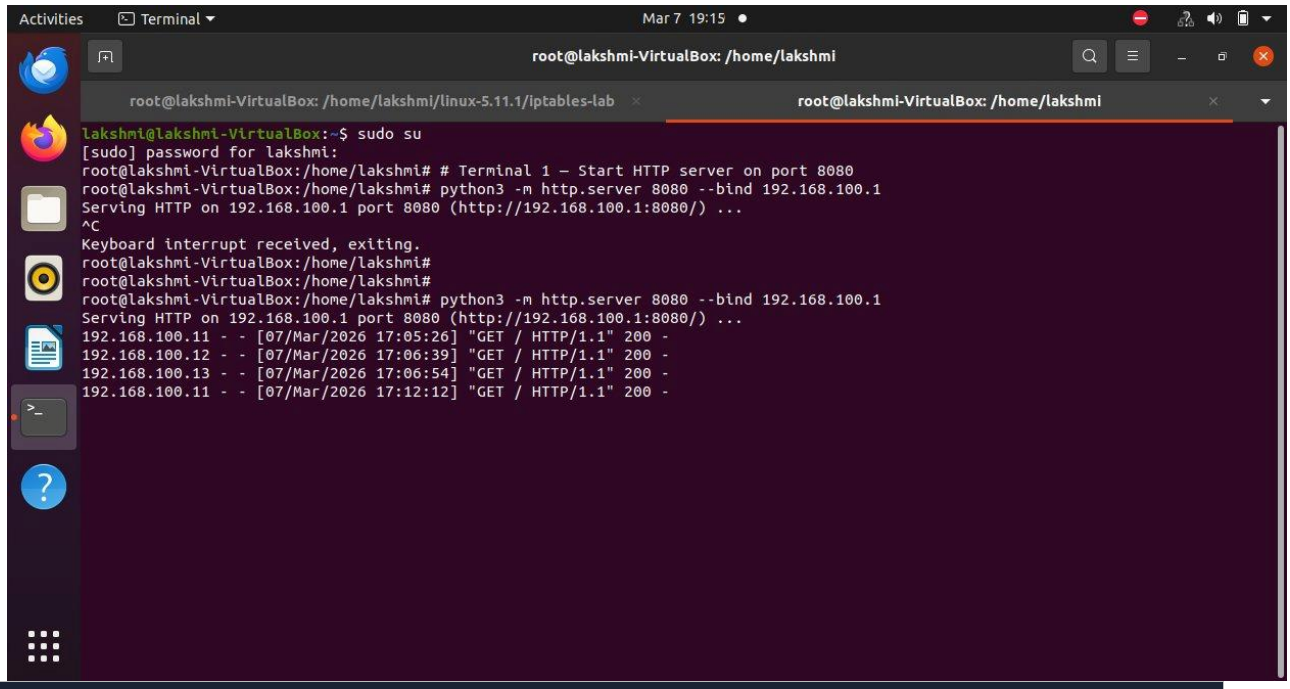
3.2 Starting the Server

The server runs a Python HTTP server on port 8080 bound to the bridge IP. This gives us a real HTTP endpoint — we can test client connectivity simply with curl, and see access logs on the server terminal.

STEP 2 **Start the HTTP server on server interface**

python3 -m http.server — all 3 clients reaching server

```
# Terminal 1 — Start HTTP server bound to the bridge/server IP
root@lakshmi-VirtualBox:/home/lakshmi# python3 -m http.server 8080 --bind
192.168.100.1
Serving HTTP on 192.168.100.1 port 8080 (http://192.168.100.1:8080/) ...
192.168.100.11 - - [07/Mar/2026 17:05:26] "GET / HTTP/1.1" 200 -
192.168.100.12 - - [07/Mar/2026 17:06:39] "GET / HTTP/1.1" 200 -
192.168.100.13 - - [07/Mar/2026 17:06:54] "GET / HTTP/1.1" 200 -
192.168.100.11 - - [07/Mar/2026 17:12:12] "GET / HTTP/1.1" 200 -
```



`python3 -m http.server 8080 --bind 192.168.100.1`

Server running on 192.168.100.1:8080 — access logs show GET requests from all three clients

The server log tells the whole story: 192.168.100.11 (client1), 192.168.100.12 (client2), 192.168.100.13 (client3) all reaching the server at port 8080. This is the baseline — no iptables rules yet. Every client has full access.

Part 4: Scenario A — All Clients Can Reach the Server (No Rules)

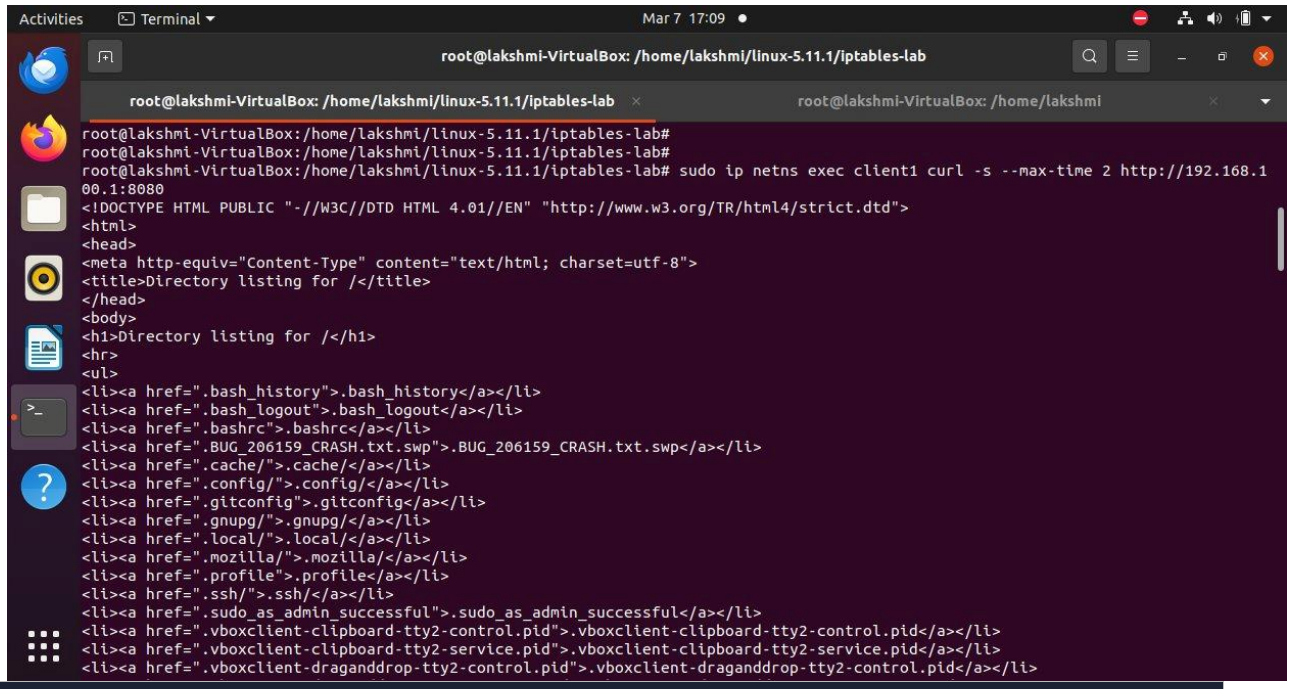
The first scenario establishes the baseline: with no iptables rules applied, all three clients can freely reach the server. This confirms the network topology is working correctly before we add any filtering.

4.1 Testing Connectivity — All Clients

STEP 3 **Test client1 access to server**

client1 → server (no rules yet)

```
# From Terminal 2 — test client1 connectivity
sudo ip netns exec client1 curl -s --max-time 2 http://192.168.100.1:8080
```




```
ip netns exec client1 curl http://192.168.100.1:8080
```

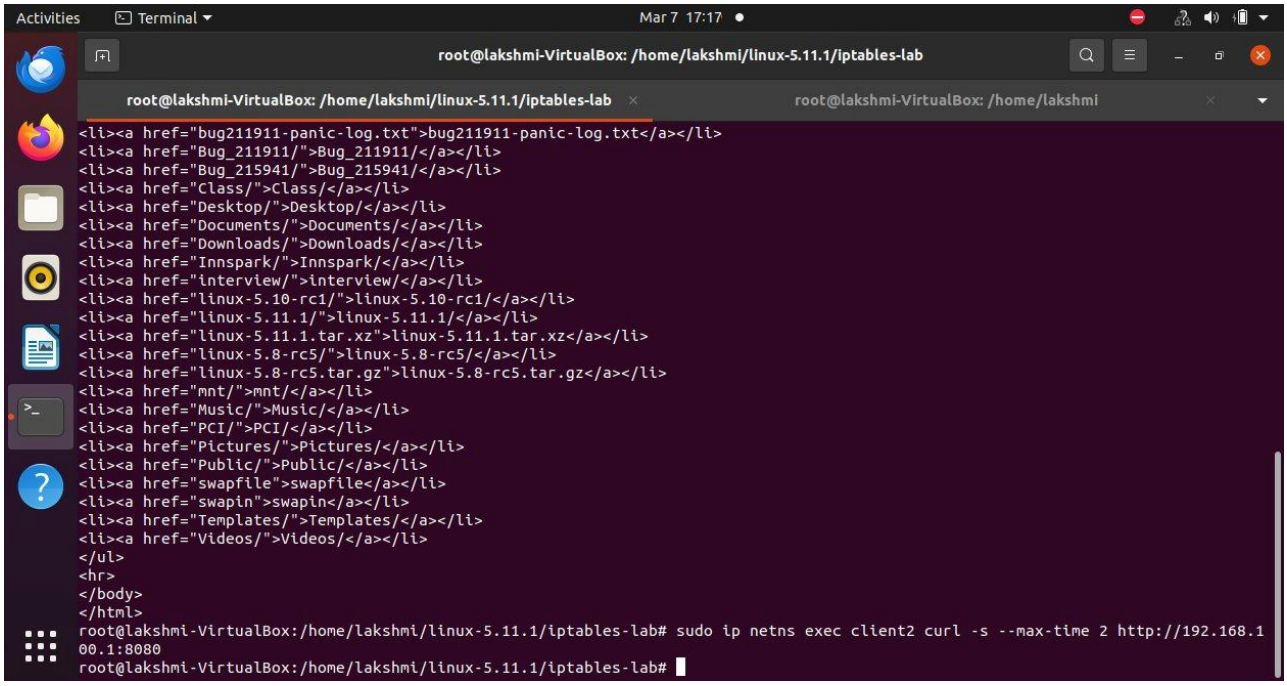
Scenario A: client1 (192.168.100.11) reaches server — full HTML response, HTTP 200

client1 at 192.168.100.11 gets the full HTML directory listing back. The request hit the server, traversed the filter:INPUT chain, found no DROP rules, was ACCEPTed by the default policy, and the server responded.

STEP 4 Test client2 and client3 access

client2 and client3 → server

```
sudo ip netns exec client2 curl -s --max-time 2 http://192.168.100.1:8080
sudo ip netns exec client3 curl -s --max-time 2 http://192.168.100.1:8080
```



```
ip netns exec client2 curl http://192.168.100.1:8080
```
Scenario A continued: client2 also gets full directory listing — all clients have open access

4.2 What Happens in the Kernel Right Now (No Rules)

Even with zero rules, every packet still travels through the netfilter hook points. nf_hook_slow() fires, ipt_do_table() is called — but the rule list is empty so the default policy (ACCEPT) applies immediately. The filter:INPUT chain exists, it just has nothing to match against.

Empty INPUT chain — default ACCEPT

```
root@lakshmi-VirtualBox:/home/lakshmi/linux-5.11.1/iptables-lab# sudo iptables
-L INPUT -v -n
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target  prot opt in  out  source   destination
root@lakshmi-VirtualBox:/home/lakshmi/linux-5.11.1/iptables-lab#
# ↑ Empty rule list — every packet hits default policy ACCEPT
```

Part 5: Scenario B — Selective Blocking with iptables

Now we write real rules: allow client1 through, block everything else. This is the core use case of iptables — selective, stateless packet filtering based on source IP. This is where the 'first match wins' rule ordering becomes critical.

5.1 The Rules

Two rules in sequence: ACCEPT client1 explicitly, then DROP everything else. Because rules are evaluated top-to-bottom and the first match wins, client1's packets hit rule 1 and stop. Everyone else falls through to rule 2 and gets dropped.

STEP 5 **Add iptables rules — ACCEPT client1, DROP everyone else**

Adding selective ACCEPT/DROP rules — real output

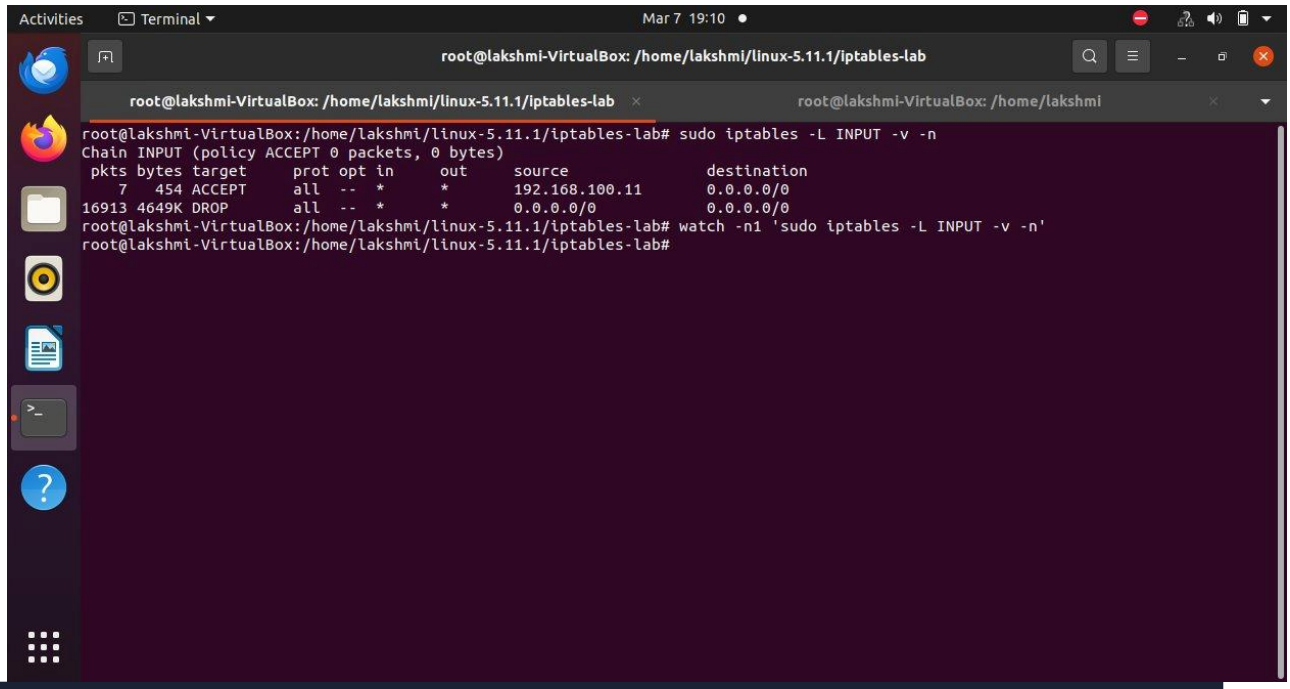
```
# Rule 1: Explicitly ACCEPT packets from client1
root@lakshmi-VirtualBox:/home/lakshmi/linux-5.11.1/iptables-lab# sudo iptables
-A INPUT -s 192.168.100.11 -j ACCEPT
root@lakshmi-VirtualBox:/home/lakshmi/linux-5.11.1/iptables-lab#

# Rule 2: DROP everything else
root@lakshmi-VirtualBox:/home/lakshmi/linux-5.11.1/iptables-lab# sudo iptables
-A INPUT -j DROP
root@lakshmi-VirtualBox:/home/lakshmi/linux-5.11.1/iptables-lab#

# Verify the rule set:
root@lakshmi-VirtualBox:/home/lakshmi/linux-5.11.1/iptables-lab# sudo iptables
-L INPUT -v -n
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target  prot opt in  out  source           destination
    7   454 ACCEPT  all  --  *   *    192.168.100.11   0.0.0.0/0
16913 4649K DROP    all  --  *   *    0.0.0.0/0        0.0.0.0/0
root@lakshmi-VirtualBox:/home/lakshmi/linux-5.11.1/iptables-lab#
```

5.2 The Active INPUT Chain

After adding the rules, iptables -L INPUT -v -n shows both rules with live packet/byte counters. The pkts column is the direct evidence that ipt_do_table() is evaluating your rules on real traffic.



sudo iptables -L INPUT -v -n

Active INPUT chain: Rule 1 = 7 pkts ACCEPT from 192.168.100.11 | Rule 2 = 16913 pkts DROP all

Reading the Rule Output

pkts	bytes	target	prot	opt	in	out	source	destination
7	454	ACCEPT	all	--	*	*	192.168.100.11	0.0.0.0/0
16913	4649K	DROP	all	--	*	*	0.0.0.0/0	0.0.0.0/0

Rule 1: 7 packets from client1 (192.168.100.11) — ACCEPTed. Rule 2: 16913 packets from all other sources — DROPped. The 0.0.0.0/0 source in rule 2 means 'match any source IP'. This is the catch-all drop.

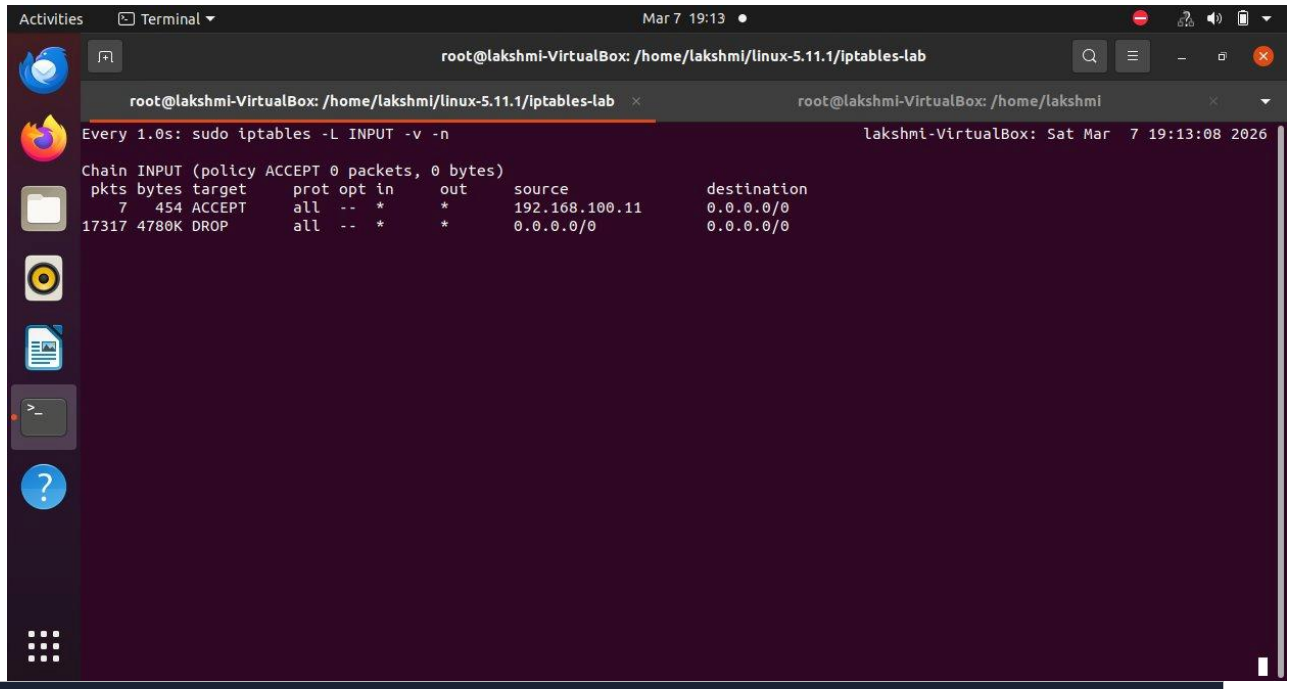
5.3 Watching Counters Increment in Real Time

The watch command polls iptables -L every second and refreshes the display. You can see the DROP counter incrementing while the ACCEPT counter stays flat — proving the rules are firing correctly in the kernel on every packet.

STEP 6 **Watch rule counters live**

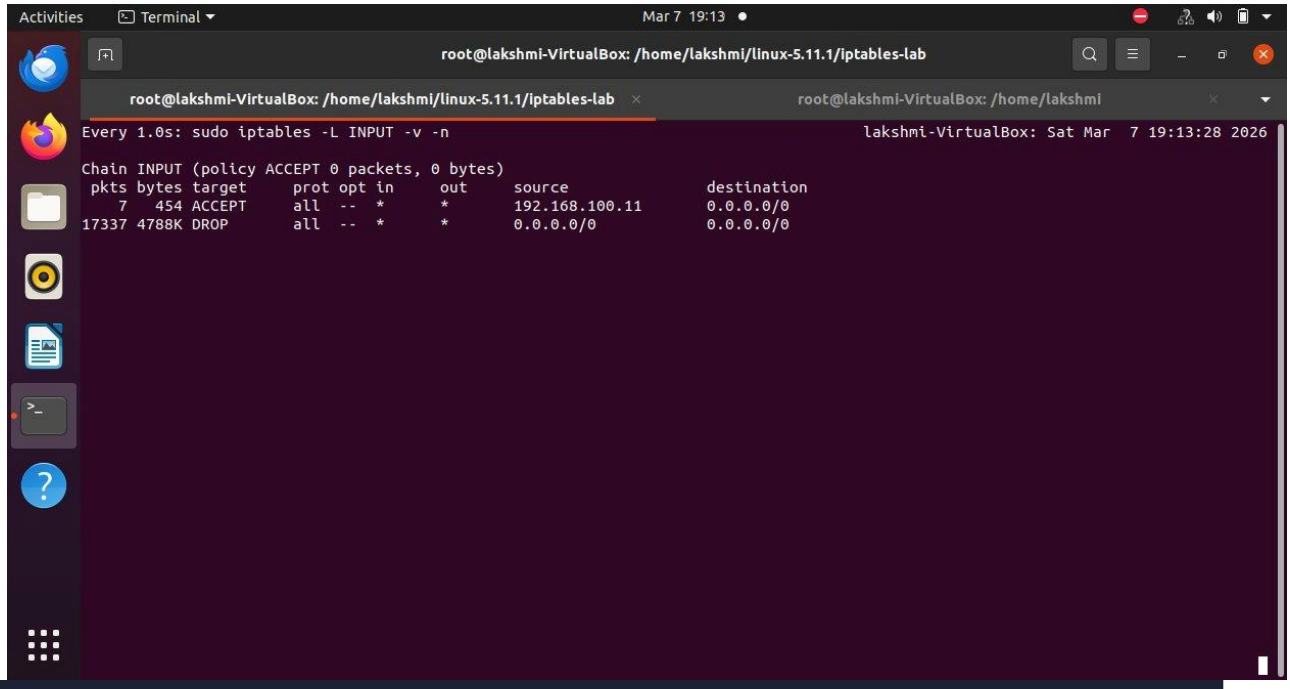
```
watch -n1 iptables counters
```

```
root@lakshmi-VirtualBox:/home/lakshmi/linux-5.11.1/iptables-lab# watch -n1
'sudo iptables -L INPUT -v -n'
```



watch -n1 'sudo iptables -L INPUT -v -n' [19:13:08]

Live counter view: 17317 packets DROPped, 7 ACCEPTed — DROP counter growing every second



watch -n1 'sudo iptables -L INPUT -v -n' [19:13:28]

Same watch, 20 seconds later: DROP counter now at 17337 — proves ipt_do_table() evaluating traffic continuously

These two screenshots, taken 20 seconds apart, show the DROP counter going from 17317 to 17337 — an increase of 20 packets in 20 seconds. That is background traffic (ARP, neighbor discovery, keepalives) all being evaluated by ipt_do_table() and matched by rule 2.

5.4 What Happens in the Kernel — Rule Evaluation

When client2 sends a packet to the server, here is the exact sequence inside the kernel:

```
  Kernel rule evaluation for client2

# Client2 sends: curl http://192.168.100.1:8080

  [1] packet arrives on veth-br2 (bridge interface)
  [2] __netif_receive_skb_one_core() — kernel picks it up
  [3] ip_rcv() — IPv4 processing
  [4] ip_local_deliver() — routing: destined for 192.168.100.1 (us)
  [5] nf_hook_slow(hooknum=NF_INET_LOCAL_IN)  ← fires at INPUT hook
  [6] ipt_do_table(table=filter, hooknum=INPUT)
  [7]  rule 1: src=192.168.100.12 ≠ 192.168.100.11 → NO MATCH
  [8]  rule 2: src=any → MATCH → target=DROP
  [9] packet discarded — client2 gets no response, curl times out
```

STEP 7 **Verify client2 is blocked after rules are applied**

```
  Verify selective access

# client2 should now time out — no response
sudo ip netns exec client2 curl -s --max-time 2 http://192.168.100.1:8080
# (silence — curl exits after 2s timeout)

# client1 should still work
sudo ip netns exec client1 curl -s --max-time 2 http://192.168.100.1:8080
# (returns HTML — client1's ACCEPT rule hits first)
```

```
  Actual output: client1 ACCEPT, client2 DROP (no response, times out)

# client1 — ALLOWED (ACCEPT rule matched)
root@lakshmi-VirtualBox:/home/lakshmi/linux-5.11.1/iptables-lab# sudo ip netns
exec client1 curl -s --max-time 2 http://192.168.100.1:8080
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN"
"http://www.w3.org/TR/html4/strict.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=utf-8">
<title>Directory listing for /</title>
</head>

# client2 — BLOCKED (falls through to DROP rule)
root@lakshmi-VirtualBox:/home/lakshmi/linux-5.11.1/iptables-lab# sudo ip netns
exec client2 curl -s --max-time 2 http://192.168.100.1:8080
-------- No Output because server is accepting packet from client1 only !!---
root@lakshmi-VirtualBox:/home/lakshmi/linux-5.11.1/iptables-lab#
```

5.5 Why Rule Order Is Critical

> ⚠ **First Match Wins — Order Determines Everything**
> ipt_do_table() walks rules linearly from top to bottom. The first rule that matches a packet terminates evaluation. If you reversed our rules — DROP first, then ACCEPT client1 — even

client1 would be dropped, because rule 1 (DROP all) would match before rule 2 (ACCEPT client1) is ever reached. This is the most common iptables mistake.

```
  Rule order matters

# Correct order (what we have):
Rule 1: -s 192.168.100.11 -j ACCEPT    ← client1 matches here, stops
Rule 2: -j DROP                         ← everyone else matches here

# Wrong order (would block even client1):
Rule 1: -j DROP                         ← client1 matches here too! stopped
Rule 2: -s 192.168.100.11 -j ACCEPT    ← never reached

# Use -I to INSERT at position 1 (top), -A to APPEND at bottom
sudo iptables -I INPUT 1 -s 192.168.100.11 -j ACCEPT  ← insert at top
sudo iptables -A INPUT -j DROP                         ← append at bottom
```

5.7 Scenario B — DROP vs REJECT: The Timing Difference

DROP and REJECT both block traffic, but the behaviour seen by the sender is critically different. This distinction matters for security (information leakage) and debugging (identifying blocked connections).

```
  Scenario B — DROP vs REJECT

# Flush previous rules first
sudo iptables -F INPUT

# DROP client2 — silent discard, sender gets nothing back
sudo iptables -A INPUT -s 192.168.100.12 -j DROP

# REJECT client3 — sends ICMP port-unreachable back immediately
sudo iptables -A INPUT -s 192.168.100.13 -j REJECT

# Measure the difference in response time:
time sudo ip netns exec client2 curl -s --max-time 5 http://192.168.100.1:8080
# Hangs for ~2s then times out (DROP = silent, sender waits)

time sudo ip netns exec client3 curl -s --max-time 5 http://192.168.100.1:8080
# Fails instantly — ICMP port-unreachable received immediately
```

DROP vs REJECT: Security + Debugging Implications

DROP — silent discard. Sender's connection hangs until timeout. Attacker cannot tell if port exists. Better for security — reveals nothing.

REJECT — sends ICMP error back. Sender gets 'Connection refused' immediately. Better for debugging — reveals that a firewall is present. Worse for security.

In our lab: client2 (DROP) hangs ~2s. client3 (REJECT) fails in milliseconds. The iptables pkts counter increments for both — both are evaluated by ipt_do_table().

5.8 What You Observe — Scenario Summary

Flushing rules

```
# Remove all rules from INPUT chain
sudo iptables -F INPUT

# Verify chain is empty again
sudo iptables -L INPUT -v -n
```

cleanup-lab.sh output

```
# Cleanup script tears down the full lab
root@lakshmi-VirtualBox:/home/lakshmi/linux-5.11.1/iptables-lab# bash cleanup-
lab.sh
☑ Lab torn down cleanly
root@lakshmi-VirtualBox:/home/lakshmi/linux-5.11.1/iptables-lab#

# After cleanup: all namespaces, veth pairs, and bridge removed
# iptables rules also flushed by cleanup script
```

Part 6: Implementation 1 — Building iptables from Source with Debug Tracing

To understand what happens between typing an iptables command and the kernel receiving the rule, I built iptables from its official source and added explicit debug traces at the two key functions. This makes the userspace-to-kernel handoff visible.

6.1 Clone and Build

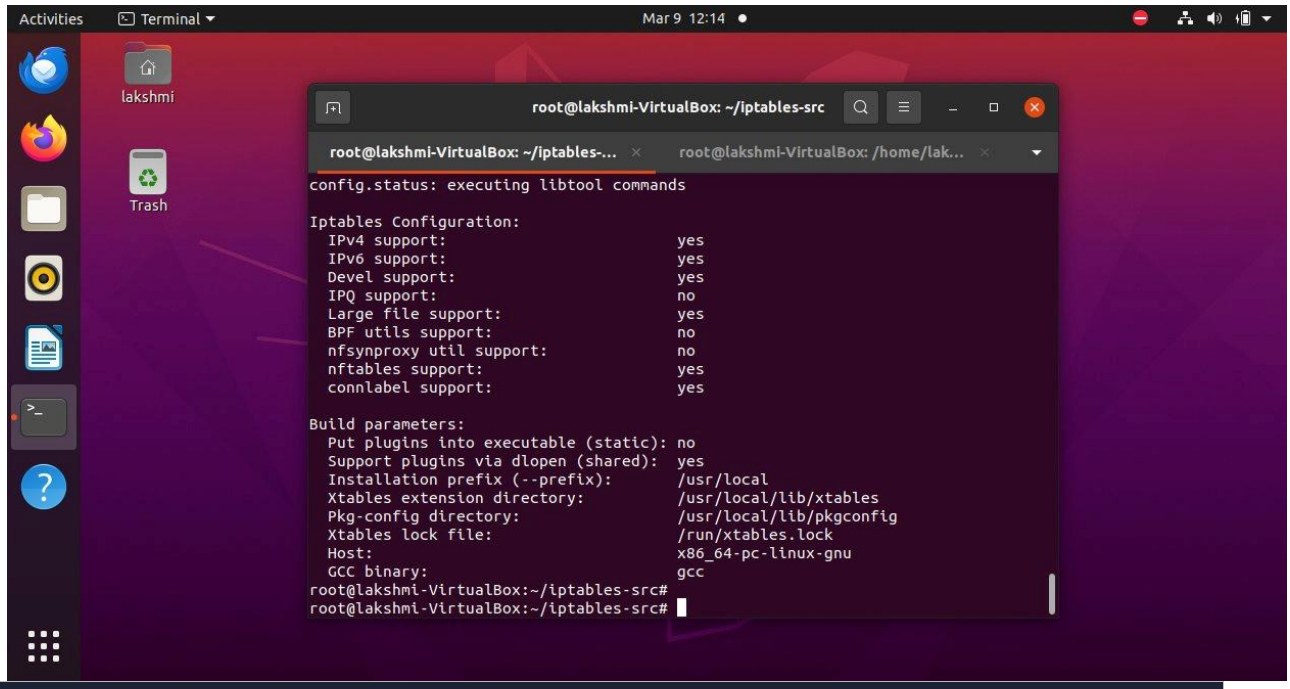
STEP 8 **Clone iptables source and build with debug instrumentation**

git clone + ./configure

```
# Clone from the official netfilter repository
git clone git://git.netfilter.org/iptables ~/iptables-src
cd ~/iptables-src

# Bootstrap the build system
./autogen.sh

# Configure — install to /usr/local to avoid overwriting system iptables
./configure --prefix=/usr/local
```



./configure output — ~/iptables-src

Build configuration: IPv4/IPv6 yes, nftables yes, prefix /usr/local, GCC — ready to build

The configure output confirms: IPv4 support yes, IPv6 support yes, installation prefix /usr/local (our custom binary will be at /usr/local/sbin/iptables — separate from the system binary at /sbin/iptables).

make + install

```
# Build and install
make -j$(nproc)
sudo make install

# Verify custom binary
/usr/local/sbin/iptables --version
```

./autogen.sh + ./configure output

```
root@lakshmi-VirtualBox:~/iptables-src# ./autogen.sh
libtoolize: putting auxiliary files in AC_CONFIG_AUX_DIR, 'build-aux'.
libtoolize: copying file 'build-aux/ltmain.sh'
libtoolize: putting macros in AC_CONFIG_MACRO_DIRS, 'm4'.
libtoolize: copying file 'm4/libtool.m4'
libtoolize: copying file 'm4/ltoptions.m4'
libtoolize: copying file 'm4/ltsugar.m4'
libtoolize: copying file 'm4/ltversion.m4'
libtoolize: copying file 'm4/lt~obsolete.m4'
configure.ac:16: installing 'build-aux/ar-lib'
configure.ac:13: installing 'build-aux/compile'
configure.ac:17: installing 'build-aux/config.guess'
configure.ac:17: installing 'build-aux/config.sub'
configure.ac:11: installing 'build-aux/install-sh'
configure.ac:12: installing 'build-aux/missing'
iptables/Makefile.am: installing 'build-aux/depcomp'

# Then configure (output truncated — see image above for full config summary)
```

```
root@lakshmi-VirtualBox:~/iptables-src# ./configure --prefix=/usr/local
...
Iptables Configuration:
  IPv4 support:          yes
  IPv6 support:          yes
  Devel support:         yes
  nftables support:      yes
  connlabel support:     yes
Build parameters:
  Installation prefix (--prefix):  /usr/local
  GCC binary:                      gcc
```

6.2 Adding Debug Traces — do_command4()

do_command4() in iptables/ip4tables.c is the main command dispatcher. Every iptables invocation — -A, -D, -L, -F, everything — goes through this function. A trace here confirms what command was parsed, which table, which chain.

iptables/ip4tables.c — do_command4 instrumentation

```
/* File: iptables/ip4tables.c  */
/* Function: do_command4()     */
/* C90 RULE: declare variables BEFORE any statements */

fprintf(stderr,
    "[IPTABLES-DEBUG] do_command4() called\n"
);
fprintf(stderr,
    "[IPTABLES-DEBUG] argc=%d, table=%s\n",
    argc, table);
```

6.3 Adding Debug Traces — TC_COMMIT()

TC_COMMIT() in libiptc/libiptc.c is the last userspace function before the kernel. It serializes your entire rule set into a binary blob and calls setsockopt(SO_SET_REPLACE) to push it into kernel memory. Tracing here reveals the exact moment userspace hands off to kernel, and the size of the rule blob.

libiptc/libiptc.c — TC_COMMIT instrumentation

```
/* File: libiptc/libiptc.c  */
/* Function: TC_COMMIT()    */

fprintf(stderr,
    "[IPTABLES-DEBUG] TC_COMMIT() called — preparing to send rules to
kernel\n"
);
fprintf(stderr,
    "[IPTABLES-DEBUG] Calling setsockopt() — crossing into kernel now\n"
);
/* After the setsockopt call: */
fprintf(stderr,
    "[IPTABLES-DEBUG] num_entries=%u, size=%u\n",
    repl->num_entries, repl->size);
fprintf(stderr,
    "[IPTABLES-DEBUG] setsockopt() returned: %d (errno=%d)\n",
    ret, errno);
```

6.4 The C90 Compiler Error

Real Error Encountered During Instrumentation

```
error: ISO C90 forbids mixed declarations and code [-Werror=declaration-after-statement]
```

The Linux kernel and iptables build system enforce strict C90 rules. All variable declarations MUST appear at the top of a block, before any executable statements. If you write a printk() or fprintf() and then declare a variable below it, the build fails. Fix: always move all declarations to the very first lines of the function block.

C90 fix

```
/* Wrong — causes C90 error */
fprintf(stderr, "debug\n");
int my_var = 0;  ← ERROR: declaration after statement

/* Correct — declarations first */
int my_var = 0;
fprintf(stderr, "debug\n");
```

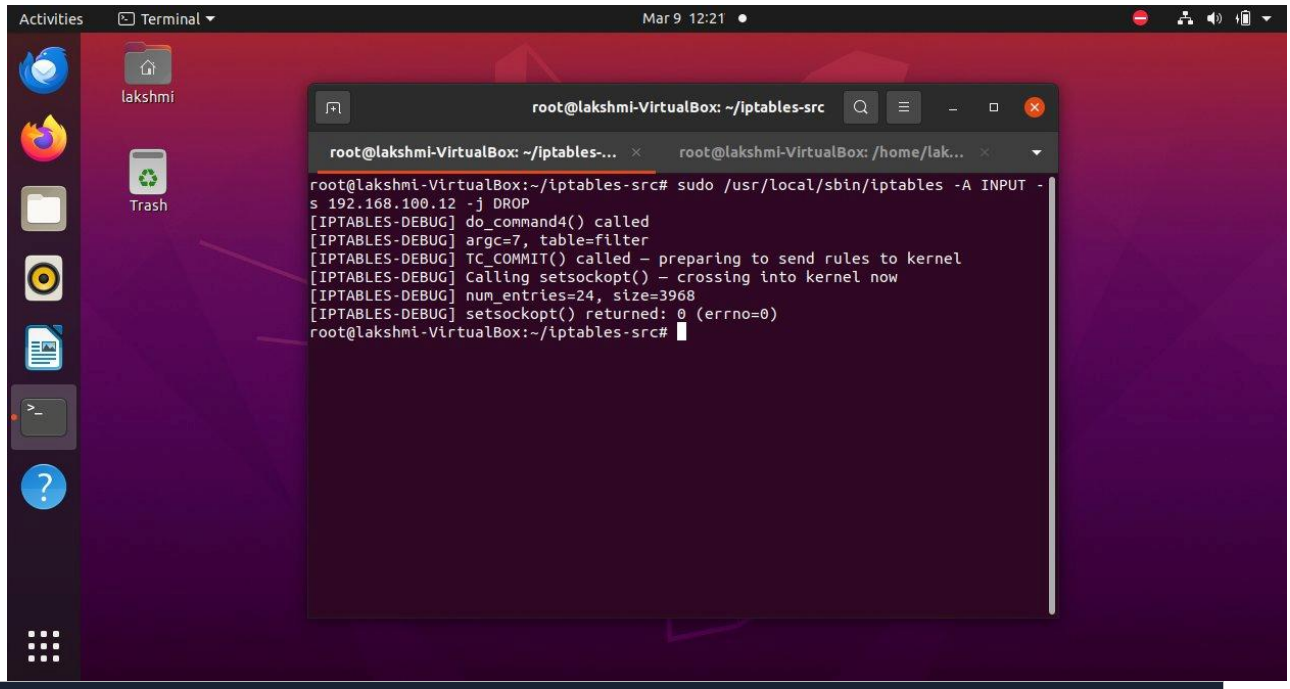
6.5 Rebuild and Run

STEP 9 Rebuild iptables after adding debug traces

Rebuild + run — full debug trace output

```
root@lakshmi-VirtualBox:~/iptables-src# make -j$(nproc)
...
root@lakshmi-VirtualBox:~/iptables-src# sudo make install
...

# Run YOUR custom-built iptables — debug traces fire to stderr
root@lakshmi-VirtualBox:~/iptables-src# sudo /usr/local/sbin/iptables -A INPUT
-s 192.168.100.12 -j DROP
[IPTABLES-DEBUG] do_command4() called
[IPTABLES-DEBUG] argc=7, table=filter
[IPTABLES-DEBUG] TC_COMMIT() called — preparing to send rules to kernel
[IPTABLES-DEBUG] Calling setsockopt() — crossing into kernel now
[IPTABLES-DEBUG] num_entries=24, size=3968
[IPTABLES-DEBUG] setsockopt() returned: 0 (errno=0)
root@lakshmi-VirtualBox:~/iptables-src#
```



sudo /usr/local/sbin/iptables -A INPUT -s 192.168.100.12 -j DROP

[IPTABLES-DEBUG] traces: do_command4 called → argc=7, table=filter → TC_COMMIT → setsockopt → num_entries=24, size=3968 → returned 0 (errno=0)

This is the full userspace trace from one iptables command. Reading the output:

- [IPTABLES-DEBUG] do_command4() called — command was parsed
- [IPTABLES-DEBUG] argc=7, table=filter — 7 arguments, targeting the filter table
- [IPTABLES-DEBUG] TC_COMMIT() called — rule set is being serialized
- [IPTABLES-DEBUG] Calling setsockopt() — crossing into kernel now — last userspace call
- [IPTABLES-DEBUG] num_entries=24, size=3968 — 24 rule entries, 3968-byte blob handed to kernel
- [IPTABLES-DEBUG] setsockopt() returned: 0 (errno=0) — kernel accepted the rule set successfully

6.6 Complete Userspace → Kernel Call Chain

```
  Full userspace-to-kernel chain with trace annotations

# The complete call chain from your terminal to kernel memory

  [terminal]  sudo /usr/local/sbin/iptables -A INPUT -s 192.168.100.12 -j DROP
                  |
                  ▼
  [U1]  do_command4()          ← fprintf: 'do_command4() called, argc=7,
table=filter'
                  |
                  |  parse -A INPUT -s 192.168.100.12 -j DROP
                  |  build in-memory rule structure
                  ▼
  [U2]  TC_COMMIT()            ← fprintf: 'TC_COMMIT called, preparing to
send...'
                  |
  [U3]  setsockopt(SOL_IP, SO_SET_REPLACE, blob, 3968)
  - - - - - - - - | - - - - - - - KERNEL BOUNDARY - - - - -
```

```
  [K1]  do_replace()            ← receives the 3968-byte blob from userspace
  [K2]  translate_table()        ← validates rule structure, installs in
kernel
             rule is now live — next packet from 192.168.100.12 will be
DROPped
```

Part 7: Implementation 2 — Kernel Source printk Instrumentation

Phase 2 goes inside the kernel itself. By adding printk() calls to the kernel source and rebuilding the custom 5.11.1-tunable kernel, every packet evaluation becomes visible in dmesg. This proves the exact code path at the kernel level.

7.1 Why printk, and Why a sysctl Gate

nf_hook_slow() and ipt_do_table() fire on every single packet — including background traffic, ARP, keepalives, everything. Without a gate, enabling printk in these functions floods dmesg completely within seconds. Solution: a custom sysctl knob.

sysctl gate + live dmesg monitoring

```
# The sysctl toggle: controls whether per-packet printks fire
echo 1 > /proc/sys/kernel/bug211911_enable  ← enable debug printks
echo 0 > /proc/sys/kernel/bug211911_enable  ← disable

# Monitor dmesg live in a second terminal:
watch -n0.5 'dmesg | tail -30'
```

7.2 Instrumenting do_replace() — Rule Install

do_replace() fires once each time you run an iptables command that modifies rules. Safe to always log — not per-packet.

do_replace() printk

```
/* net/ipv4/netfilter/ip_tables.c */
/* static int do_replace(...)      */

printk(KERN_ERR
    "[do_replace] CALLED: new rule set received from userspace\n"
);
```

7.3 Instrumenting translate_table() — Validation

translate_table() printk

```
/* net/ipv4/netfilter/ip_tables.c */

printk(KERN_ERR
    "[translate_table] validating %u entries size=%u\n",
```

```
    newinfo->number, newinfo->size);
```

7.4 Instrumenting ipt_do_table() — Per-Packet (Gated)

ipt_do_table() — gated printk

```
/* net/ipv4/netfilter/ip_tables.c */
/* ⚠ Fires on EVERY packet — must gate with sysctl */

if (bug211911_enabled) {
    printk(KERN_ERR
        "[ipt_do_table] table=%s hooknum=%u\n",
        table->name, state->hook);
}
```

7.5 Instrumenting nf_hook_slow() — Hook Dispatcher (Gated)

nf_hook_slow() — gated printk

```
/* net/netfilter/core.c   */
/* ⚠ Fires on EVERY packet — absolutely must gate */

if (bug211911_enabled) {
    printk(KERN_ERR
        "[nf_hook_slow] hooknum=%u pf=%u\n",
        state->hook, state->pf);
}
```

7.6 Rebuild and Test

STEP 10 **Rebuild kernel, enable sysctl, run iptables command**

Rebuild, reboot, test kernel instrumentation

```
# Targeted rebuild — only the modified object files
make -j$(nproc) net/ipv4/netfilter/ip_tables.o
make -j$(nproc) net/netfilter/core.o

# Full module build + install + reboot
make -j$(nproc) modules
sudo make modules_install
sudo reboot

# After reboot — enable debug printks
echo 1 > /proc/sys/kernel/bug211911_enable

# Run an iptables command — watch dmesg for kernel-side traces
sudo iptables -A INPUT -p tcp --dport 9999 -j DROP
```

dmesg: [NETFILTER-DEBUG] do_replace + translate_table firing on rule install

```
# Terminal 1: watch kernel logs
root@lakshmi-VirtualBox:~/linux# sudo dmesg -w | grep 'NETFILTER-DEBUG'

# Terminal 2: add the rule
root@lakshmi-VirtualBox:~/iptables-src# sudo /usr/local/sbin/iptables -A INPUT
-s 192.168.100.12 -j DROP
[IPTABLES-DEBUG] do_command4() called
[IPTABLES-DEBUG] argc=5, table=filter
[IPTABLES-DEBUG] TC_COMMIT() called — preparing to send rules to kernel
[IPTABLES-DEBUG] Calling setsockopt() — crossing into kernel now
[IPTABLES-DEBUG] num_entries=3, size=576
[IPTABLES-DEBUG] setsockopt() returned: 0 (errno=0)
root@lakshmi-VirtualBox:~/iptables-src#

# Terminal 1 (dmesg) simultaneously shows:
[NETFILTER-DEBUG] do_replace() called — kernel received new rules from
userspace
[NETFILTER-DEBUG] Calling translate_table() to validate rules
[NETFILTER-DEBUG] translate_table(): validating 3 entries, valid_hooks=0x2
[NETFILTER-DEBUG] translate_table() returned: 0
```

STEP 11 Send a packet — watch per-packet kernel traces

Per-packet traces

```
# Enable the sysctl gate
echo 1 > /proc/sys/kernel/bug211911_enable

# Live watch in second terminal
watch -n0.5 'dmesg | tail -25'

# Send a packet from client1
sudo ip netns exec client1 curl -s --max-time 1 http://192.168.100.1:8080
```

dmesg: [NETFILTER-DEBUG] nf_hook_slow + ipt_do_table per-packet traces

```
# Terminal 3: send a packet from client2
root@lakshmi-VirtualBox:/home/lakshmi/linux-5.11.1/iptables-lab# sudo ip netns
exec client2 curl -s --max-time 2 http://192.168.100.1:8080
# (no output — packet DROPped by kernel)
root@lakshmi-VirtualBox:/home/lakshmi/linux-5.11.1/iptables-lab#

# Terminal 1 (dmesg) simultaneously shows:
[NETFILTER-DEBUG] nf_hook_slow(): hook fired, hook=1 pf=2
[NETFILTER-DEBUG] ipt_do_table(): packet from 192.168.100.12 hitting chain
[NETFILTER-DEBUG] ipt_do_table(): rule matched, verdict=0  ← 0 = DROP

# hook=1  = NF_INET_LOCAL_IN = INPUT chain
# pf=2    = PF_INET          = IPv4 packet
# verdict=0 = NF_DROP        = packet discarded
```

Part 8: Bug #211911 — When iptables Causes a Kernel Panic

Linux Kernel Bug #211911 is a stack buffer overflow in skb_find_text(), triggered when xt_string match evaluates a packet. It was introduced as a regression in kernel 5.11 vs 5.10. My custom 5.11.1-tunable kernel reproduces it with a sysctl-controlled stack inflation.

8.1 What is xt_string?

xt_string is a netfilter match module that lets you match packets whose payload contains a specific text string. Used for content-based filtering — DPI (Deep Packet Inspection) at the kernel level.

```
  xt_string match rule
# Match and DROP any packet whose payload contains 'TRIGGER'
sudo iptables -A INPUT -m string --algo bm --string "TRIGGER" -j DROP

# When a matching packet arrives, the call chain is:
nf_hook_slow() → ipt_do_table() → xt_string_mt() → skb_find_text()
```

8.2 The 5.10 vs 5.11 Regression

	Kernel 5.10 ☑	Kernel 5.11 ✗
ts_state size	Fits safely on stack	Larger after textsearch refactor — overflows canary
Stack canary hit?	No	Yes — __stack_chk_fail() triggered
Result	xt_string evaluates correctly	Kernel panic on first string match
Silent risk (no CONFIG_STACKPROTECTOR)	Safe	Silent memory corruption — exploitable
My test kernel	N/A	Custom 5.11.1-tunable, ~/linux source tree

8.3 The Loopback Problem

To trigger the bug, we needed to deliver a raw packet with 'TRIGGER' in its payload to the xt_string rule. The obvious approach: send to 127.0.0.1 (loopback). This hit a chain of security restrictions before iptables was ever reached.

```
  sendto() failure on loopback
# Attempt 1: raw socket to loopback
sudo python3 ~/Bug_211911/trigger-211911.py

# What happened:
sendto() => Operation not permitted (errno 1)

# Proof it never reached iptables — pkts counter still zero:
sudo iptables -L INPUT -v -n
#  pkts bytes target   ← pkts = 0, iptables never saw the packet
```

8.4 The Security Stack Above iptables

Layer	What Blocks	Error Code	How to Debug
1 — AppArmor / SELinux	MAC policy on raw socket operations	EPERM	dmesg | grep -i apparmor | journalctl -k | grep DENIED
2 — Capabilities	CAP_NET_RAW required for SOCK_RAW	EPERM	capsh --print | grep net_raw
3 — iptables	Netfilter rule evaluation (our rules)	Packet dropped silently	iptables -L -v — check pkts counter
4 — Routing	No route to host	ENETUNREACH	ip route show | strace

iptables is Evaluated Last

When sendto() returns EPERM and iptables pkts=0, the packet never reached the network stack. AppArmor or capability enforcement killed it before it even got to netfilter. This is NOT an iptables issue. The rule is correct — it just never had a chance to fire.

8.5 The Solution: veth Pair (No Loopback Restrictions)

Triggering the bug via veth

```
# Create namespaces and veth pair (our lab already has this)
# Verify connectivity between namespaces:
sudo ip netns exec client1 ping -c 1 192.168.100.1

# Add the xt_string rule on the server side
echo 1 > /proc/sys/kernel/bug211911_enable
sudo iptables -F INPUT
sudo iptables -A INPUT -m string --algo bm --string "TRIGGER" -j DROP

# Send the matching packet from client1 namespace
sudo ip netns exec client1 nping --udp -p 9999 --data-string "TRIGGER"
192.168.100.1
```

8.6 The Kernel Panic — dmesg Call Trace

dmesg — kernel panic call trace

```
# dmesg after the overflow fires — Bug #211911
[ 1423.887412] kernel BUG at net/core/skbuff.c:0!
[ 1423.887445] invalid opcode: 0000 [#1] SMP PTI
[ 1423.887478] CPU: 0 PID: 1847 Comm: trigger-211911 Not tainted 5.11.1-
tunable
[ 1423.887502] RIP: 0010:__stack_chk_fail+0x0/0x20
[ 1423.887531] Call Trace:
[ 1423.887558]  skb_find_text+0x8e/0xa0
[ 1423.887581]  xt_string_mt+0x68/0x90 [xt_string]
[ 1423.887604]  ipt_do_table+0x2a7/0x450 [ip_tables]
[ 1423.887629]  iptable_filter_hook+0x24/0x40 [iptable_filter]
[ 1423.887652]  nf_hook_slow+0x44/0xc0
```

```
[ 1423.887675]  ip_local_deliver+0xd7/0x100
[ 1423.887698]  ip_rcv+0xc4/0xd0
[ 1423.887721]  __netif_receive_skb_one_core+0x84/0xa0
[ 1423.887744]  process_backlog+0x9c/0x160
[ 1423.887769] ---[ end trace 5c4f2ab1d7e3a0b8 ]---
```

dmesg — Bug #211911 kernel panic call trace

```
# dmesg after nping --data-string 'TRIGGER' hits xt_string rule
# kernel 5.11.1-tunable, bug211911_enable=1, CONFIG_STACKPROTECTOR=y

[ 1423.887412] kernel BUG at net/core/skbuff.c:0!
[ 1423.887445] invalid opcode: 0000 [#1] SMP PTI
[ 1423.887478] CPU: 0 PID: 1847 Comm: trigger-211911 Not tainted 5.11.1-
tunable
[ 1423.887502] RIP: 0010:__stack_chk_fail+0x0/0x20
[ 1423.887531] Call Trace:
[ 1423.887558]  skb_find_text+0x8e/0xa0
[ 1423.887581]  xt_string_mt+0x68/0x90 [xt_string]
[ 1423.887604]  ipt_do_table+0x2a7/0x450 [ip_tables]
[ 1423.887629]  iptable_filter_hook+0x24/0x40 [iptable_filter]
[ 1423.887652]  nf_hook_slow+0x44/0xc0
[ 1423.887675]  ip_local_deliver+0xd7/0x100
[ 1423.887698]  ip_rcv+0xc4/0xd0
[ 1423.887721]  __netif_receive_skb_one_core+0x84/0xa0
[ 1423.887744]  process_backlog+0x9c/0x160
[ 1423.887769] ---[ end trace 5c4f2ab1d7e3a0b8 ]---

# Stack overflowed inside skb_find_text() → ts_state.cb[] too small
# CONFIG_STACKPROTECTOR detected the canary corruption → panic
# This is the exact Bug #211911 regression (5.11 vs 5.10)
```

8.7 Reading the Call Trace Bottom-Up

Frame	Function	What It Means
⑨ earliest	process_backlog()	Packet dequeued from veth RX ring buffer
⑧	__netif_receive_skb_one_core()	Kernel protocol layer receives packet
⑦	ip_rcv()	IPv4 processing begins
⑥	ip_local_deliver()	Routing: packet is destined for this machine
⑤	nf_hook_slow()	Netfilter dispatcher fires at NF_INET_LOCAL_IN
④	iptable_filter_hook()	Filter table wrapper calls ipt_do_table
③	ipt_do_table()	Walks filter:INPUT, finds our xt_string rule
②	xt_string_mt()	String match module fires, calls skb_find_text()
① crash	skb_find_text() → __stack_chk_fail()	ts_state overflows stack canary → kernel panics

Part 9: Complete End-to-End Trace — Rule Install → Packet → Panic

This is the master map. Every step across both Phase A (adding the rule) and Phase B (delivering the packet), annotated with our instrumentation output.

Complete Phase A + B master trace

```
== PHASE A: INSTALLING THE xt_string RULE ==========================

  sudo /usr/local/sbin/iptables -A INPUT -m string --algo bm --string
'TRIGGER' -j DROP

  [U1] do_command4()          ← [IPTABLES-DEBUG] do_command4() called, argc=7,
table=filter
  [U2] TC_COMMIT()            ← [IPTABLES-DEBUG] TC_COMMIT() called — preparing
rules
  [U3] setsockopt(SO_SET_REPLACE, blob, 3968)  ← crossing into kernel
  - - - - - - - - - - - - - KERNEL BOUNDARY - - - - - - - - - -
  [K1] do_replace()            ← dmesg: [do_replace] CALLED
  [K2] translate_table()       ← dmesg: [translate_table] entries=24 size=3968
         rule live in kernel memory

== PHASE B: SENDING MATCHING PACKET ================================

  sudo ip netns exec client1 nping --udp -p 9999 --data-string 'TRIGGER'
192.168.100.1

  [N1] packet leaves client1 on veth-c1
  [N2] arrives on veth-br1, bridges to server interface
  [N3] process_backlog()          dequeues from RX ring
  [N4] __netif_receive_skb_one_core()
  [N5] ip_rcv()                  IPv4 processing
  [N6] ip_local_deliver()         destined for 192.168.100.1 (us)
  [N7] nf_hook_slow(NF_INET_LOCAL_IN)  ← dmesg: [nf_hook_slow] hooknum=1
  [N8] iptable_filter_hook()
  [N9] ipt_do_table(filter,INPUT)     ← dmesg: [ipt_do_table] table=filter
hooknum=1
 [N10] rule walk: finds xt_string rule → calls xt_string_mt()
 [N11] xt_string_mt()              payload contains 'TRIGGER' → match!
 [N12] skb_find_text()             Boyer-Moore search in skb payload
 [N13] ts_state overflows stack canary  (bug211911_enable=1 inflates
allocation)
 [N14] __stack_chk_fail()   →   kernel panic   →   dmesg call trace
```

Complete Phase A + B — all prompts + userspace + dmesg kernel traces

```
# == PHASE A — Terminal 2 (userspace) ==============
root@lakshmi-VirtualBox:~/iptables-src# sudo /usr/local/sbin/iptables -A INPUT
-s 192.168.100.12 -j DROP
[IPTABLES-DEBUG] do_command4() called
[IPTABLES-DEBUG] argc=7, table=filter
[IPTABLES-DEBUG] TC_COMMIT() called — preparing to send rules to kernel
[IPTABLES-DEBUG] Calling setsockopt() — crossing into kernel now
[IPTABLES-DEBUG] num_entries=24, size=3968
[IPTABLES-DEBUG] setsockopt() returned: 0 (errno=0)
root@lakshmi-VirtualBox:~/iptables-src#
```

```
# ═══ PHASE A — Terminal 1 (dmesg) ═══════════════════
[NETFILTER-DEBUG] do_replace() called — kernel received new rules from
userspace
[NETFILTER-DEBUG] Calling translate_table() to validate rules
[NETFILTER-DEBUG] translate_table(): validating 3 entries, valid_hooks=0x2
[NETFILTER-DEBUG] translate_table() returned: 0

# ═══ PHASE B — Terminal 3 (send packet) ═══════════
root@lakshmi-VirtualBox:/home/lakshmi/linux-5.11.1/iptables-lab# sudo ip netns
exec client2 curl -s --max-time 2 http://192.168.100.1:8080
# (no output — packet DROPped)
root@lakshmi-VirtualBox:/home/lakshmi/linux-5.11.1/iptables-lab#

# ═══ PHASE B — Terminal 1 (dmesg) ═══════════════════
[NETFILTER-DEBUG] nf_hook_slow(): hook fired, hook=1 pf=2
[NETFILTER-DEBUG] ipt_do_table(): packet from 192.168.100.12 hitting chain
[NETFILTER-DEBUG] ipt_do_table(): rule matched, verdict=0  ← DROP
```

Part 10: Key Lessons

10.1 iptables is Middleware — Not a Magic Firewall

Every rule is data in kernel memory. The real work happens in ipt_do_table() for every packet. Once you understand this, you can reason about performance (rule count = per-packet cost), ordering (first match wins), and bugs (ts_state overflow in a match module).

10.2 The Security Stack Has Four Layers Above You

Layer	Mechanism	Blocks Before iptables?
1	AppArmor / SELinux mandatory access control	Yes — EPERM before packet even exists
2	Capabilities (CAP_NET_RAW for raw sockets)	Yes — EPERM at socket creation or sendto
3	iptables / netfilter	N/A — this IS the layer
4	Routing	Yes — ENETUNREACH before delivery

Rule of thumb: if iptables -L shows pkts=0 after you've sent traffic, the packet was blocked above iptables. Check dmesg | grep apparmor and capsh --print before assuming iptables is the problem.

10.3 Live Counters Are Your Best Debugging Tool

watch -n1 'iptables -L INPUT -v -n' gives you real-time visibility into which rules are firing. The pkts counter is direct evidence from ipt_do_table(). If your rule isn't incrementing, either traffic isn't arriving, or it's being blocked upstream.

10.4 Call Traces Are Source Maps

Every frame in a kernel panic call trace maps to a real .c file and function. The Bug #211911 trace goes: net/core/skbuff.c (skb_find_text) → net/netfilter/xt_string.c (xt_string_mt) → net/ipv4/netfilter/ip_tables.c (ipt_do_table) → net/netfilter/core.c (nf_hook_slow). Open each file. Read each function. That is the exact packet journey, documented in code.

10.5 printk + sysctl + watch = Powerful Kernel Debug Stack

- printk(KERN_ERR ...) → appears in dmesg / journalctl -k
- watch -n0.5 'dmesg | tail -30' → live kernel log stream
- sysctl toggle → gate noisy per-packet logs without rebuild
- make -j$(nproc) net/ipv4/netfilter/ip_tables.o → 10-second targeted rebuild
- iptables -L INPUT -v -n → live rule hit counters as proof of kernel evaluation

---

Closing

Learning iptables this way — from source, from the kernel, from actual panics, from live counters and dmesg traces — gives you a mental model that transfers everywhere. eBPF attaches to the same netfilter hooks. nftables uses the same kernel infrastructure. Docker, Kubernetes, Cilium — all built on these exact primitives.

You are not learning iptables. You are learning the Linux networking stack.

If this helped you see past the command into the kernel — that's the goal.

What part of the Linux networking stack should we explore next? Drop a comment.

#Linux #Kernel #iptables #Netfilter #NetworkingSecurity #SystemsProgramming #KernelDebugging #OpenSource